

Aggregazione federata: codebook ed encoder

TimeVQVAE-AD-U — Federated Analytics: le formule di merge tra client

1. Setup e notazione

Ogni client $j = 1, \dots, J$ possiede una silo di serie temporali. Un round federato parte da un codebook condiviso $e = (e_1, \dots, e_K)$, $e_k \in \mathbb{R}^D$, con K parole e dimensione D , congelato (frozen) durante il round: nessun k-means locale, tutti i client partono dallo stesso dizionario.

L'encoder mappa ogni finestra nei latenti; ciascun token latente $z \in \mathbb{R}^D$ viene assegnato alla parola più vicina (nearest-neighbour, senza gradiente sul codebook):

$$k(z) = \arg \min_{k \in \{1, \dots, K\}} \|z - e_k\|_2$$

2. Statistiche sufficienti locali (ciò che il client carica)

Sia Z_j l'insieme dei token latenti del client j nel round. Il client accumula, per ogni parola k , due statistiche decay-free (senza EMA), calcolate contro il codebook congelato:

$$n_j^k = \sum_{z \in Z_j} 1[k(z) = k] \quad m_j^k = \sum_{z \in Z_j} 1[k(z) = k] z \in \mathbb{R}^D$$

cioè n_j^k = quanti token del client j cadono nella parola k (conteggio), e m_j^k = somma vettoriale dei latenti assegnati a k . La coppia (n_j^k, m_j^k) è statistica sufficiente per il centroide della parola k : nient'altro dei dati del client serve al server.

Nel codice

```
SharedVectorQuantizer._accumulate_round_stats (model/vector_quantizer.py:203)
    round_count      += one_hot.sum(dim=0)          # n_j^k
    round_embed_sum += (ft.T @ one_hot).T          # m_j^k
```

Forzato a float32 (autocast off) affinché il merge lato-server sia il pooling ESATTO (Prop. 1).

3. Aggregazione del codebook (merge additivo — Prop. 1)

Il server NON media dei pesi (non è weight-FedAvg). Somma le statistiche sufficienti su tutti i client e ricostruisce ogni centroide. Definiti i totali aggregati:

$$N_k = \sum_{j=1}^J n_j^k \quad M_k = \sum_{j=1}^J m_j^k \quad n = \sum_{k=1}^K N_k$$

(N_k = token totali della parola k su tutta la federazione; n = token totali del round),
il nuovo peso di ogni parola del codebook è

$$e_k = \frac{M_k}{\widetilde{N}_k}, \quad \widetilde{N}_k = \frac{N_k + \varepsilon}{n + K\varepsilon} n$$

dove $\widetilde{N}_k$ è il conteggio con smoothing di Laplace ($\varepsilon \approx 1e-5$), che evita la divisione per zero sulle parole non usate e replica esattamente lo smoothing dell'EMA centralizzato (§4).

Proposizione 1 (il merge federato = M-step di k-means sul pool)

Se i dati di tutti i client fossero raccolti su un'unica macchina, e detto $Z = \cup_j Z_j$ il pool di tutti i token, l'aggiornamento del centroide (M-step di k-means) contro lo stesso codebook sarebbe:

$$e_k^{\text{pool}} = \frac{\sum_{z \in Z} 1[k(z) = k] z}{\sum_{z \in Z} 1[k(z) = k]} = \frac{\sum_j m_j^k}{\sum_j n_j^k} = \frac{M_k}{N_k}$$

Poiché la somma è associativa, M_k/N_k è indipendente da come i punti sono partizionati tra i client: per $\varepsilon \rightarrow 0$ si ha $\widetilde{N}_k \rightarrow N_k$ e quindi $e_k = e_k^{\text{pool}}$ ESATTAMENTE. Nessun bias, nessun problema di permutazione delle parole, e la comunicazione è solo (n_j, m_j) per round.

Rianimazione dei dead-code (opzionale, data-free). Le parole con $N_k < \text{soglia}$ sono 'morte'; il server le re-inizializza copiando un centroide vivo più un piccolo rumore — senza vedere dati:

$$e_k \leftarrow e_{\pi(k)} + 0.01 \xi, \quad \xi \sim \mathcal{N}(0, I), \quad \pi(k) \in \{k' : N_{k'} \geq \text{soglia}\}$$

`_server_merge` (pipeline/federated.py:320) — chiama `merge_round_stats` e poi rianima le morte.

4. Corrispondenza con l'EMA centralizzato

Il TimeVQVAE single-machine aggiorna il codebook con l'EMA di van den Oord et al. (2017):

$$N_k \leftarrow \gamma N_k + (1 - \gamma) c_k, \quad S_k \leftarrow \gamma S_k + (1 - \gamma) s_k, \quad e_k = \frac{S_k}{N_k}$$

con c_k, s_k conteggio e somma del mini-batch e decay $\gamma (=0.99)$. Il merge federato usa la STESSA forma dell'estimatore ma sostituisce lo stato EMA con i totali esatti del round (N_k, M_k) :

$$(N_k, S_k)_{\text{EMA}, \gamma} \longrightarrow (N_k, M_k)_{\text{pool esatto}}, \quad e_k = \frac{M_k}{N_k} \quad (\gamma\text{-free})$$

Stesso smoothing di Laplace, stessa divisione: il codebook federato è l'analogo esatto e senza decadimento del codebook centralizzato — questo giustifica la Proposizione 1.

5. Aggregazione dell'encoder / tokenizer (FedAvg pesato sui dati)

Un codebook condiviso ha senso solo se gli encoder concordano su cosa significhi una parola. I tensori condivisi θ dell'encoder sono mediati con il classico FedAvg (McMahan et al., 2017), pesato per la numerosità $|D_j|$ dei dati di ciascun client, poi ri-diffusi a tutti:

$$\theta = \sum_{j=1}^J \frac{|D_j|}{\sum_{i=1}^J |D_i|} \theta_j$$

Dettagli di design (FedBN / federazione parziale):

- le running-stats della BatchNorm restano LOCALI (sono statistiche dei dati, non pesi appresi);
 - in modalità 'partial' i primi blocchi front-end restano locali; si federano solo i blocchi tardivi + la proiezione verso lo spazio-codebook, così la mappa che 'vede' il codebook concorda.
- `_fedavg_encoder / _encoder_shared_keys` (pipeline/federated.py:385, 417).

In sintesi — due aggregazioni distinte

Codebook → merge ADDITIVO (statistiche sufficienti, pooling esatto — non weight-avg)

$$e_k = M_k / N_k \quad \text{con} \quad N_k = \sum_j n_j^k, \quad M_k = \sum_j m_j^k$$

Encoder → FedAvg PESATO sui dati (media dei pesi standard; BN locale = FedBN)

$$\theta = \sum_j \left(|D_j| / \sum_i |D_i| \right) \cdot \theta_j$$